processed sequentially. As a quick example, here is the same chain using `RunnableSequence.from`:

```
const sequence = RunnableSequence.from([
  RunnableLambda.from(toUpperCase),
  RunnableLambda.from(vowelCountFunction),
  prompt,
  model,
  new StringOutputParser(),
]);
```

The first approach uses `.pipe()` to chain operations sequentially, while the second uses `RunnableSequence.from()` to define all steps in a single array. The choice between them is mainly a matter of coding style preference.

Now contrast this approach with how the Vercel AI SDK typically interacts with AI models. In Vercel's architecture, API calls are often made directly to the AI model without intermediate processing steps like those seen in runnables. While Vercel allows for straightforward interactions with LLMs, it may lack the flexibility and composability that runnables provide.

Later in this chapter, we will see how to integrate this approach with the Vercel AI SDK to have the best of both worlds. Before we do that, let's explore how to manage prompts effectively using the provided `ChatPromptTemplate`.

Managing prompt templates

In the preceding listing, we saw the use of the wrapper class that creates a prompt template instead of passing a single string directly. This is an example of the *factory method* pattern, which provides a way to encapsulate the creation logic of objects since the pipe methods in the chain require a specific object instance to work.

This encapsulation allows for cleaner code and promotes reusability, as different parts of an application can generate prompts dynamically based on varying input values. For instance, different modules like customer service or marketing could use the same template structure while dynamically inserting different values for tone, length, or content based on their specific needs. The use of placeholders (e.g., with `{vowelCount}`) within the template string allows the framework to perform validation checks.

What's also useful is that the `@langchain/core/prompts` package provides a factory method for constructing few-shot learning prompts in a structured manner. Recall from the last chapter that with few-shot learning, we can enhance the performance of language models by providing them with example inputs and outputs. This technique allows the model to better understand the context and format of the desired responses.

Figure 6.2 shows the various parameters of the function `FewShotPromptTemplate` that is used for creating a prompt in this way, followed by a code listing that demonstrates its implementation.